Project 1 - Solving differential equations

Britt Haanen
(Dated: September 10, 2026)

For all codes, please find my Github repository at: *https://github.com/BrittHaanen15/Project-1-final-version.git*

PROBLEM 1

Problem a

We can show analytically that an exact solution to

$$-\frac{d^2u}{dx^2} = f(x) \text{ with } f(x) = 100e^{-10x} \tag{1}$$

is given by

$$u(x) = 1 - (1 - e^{-10})x - e^{-10x} \tag{2}$$

by taking the second derivative of Equation 2:

$$\frac{du}{dx} = -1 - e^{-10} + 10e^{-10x}$$

$$\frac{d^2u}{dx} = -100e^{-10x}$$

Thus,

$$-\frac{d^2u}{dx^2} = 100e^{-10x} = f(x) \tag{3}$$

PROBLEM 2

A C++ programme to define the vectors x and $u(x)$ was written as specified. The output of this programme was written to a .txt file and read into Python for plotting. The result is shown in Figure 1. Since I am new to C++ I plotted my result alongside a Python implementation that is natural to me. The exact match between the two confirms correctness of my C++ implementation.

PROBLEM 3

For discretisation we have to follow several steps [1, 2]. First, we discretise the continuous x values $x \in [0, 1]$ into n steps. Thus, the x values become a set of k points at spacing h called the step size. Thus

$$x_i = i * h$$

$u''(x)$ can be approximated as [2]

$$u''(x) \approx v''(x) = -\frac{v_{i+1} - 2v_i + v_{i-1}}{h^2} + O(h^2) \tag{4}$$

where the latter term is the error due to the discretised approximation as compared to the continuous function, which will shrink as the step size becomes smaller.

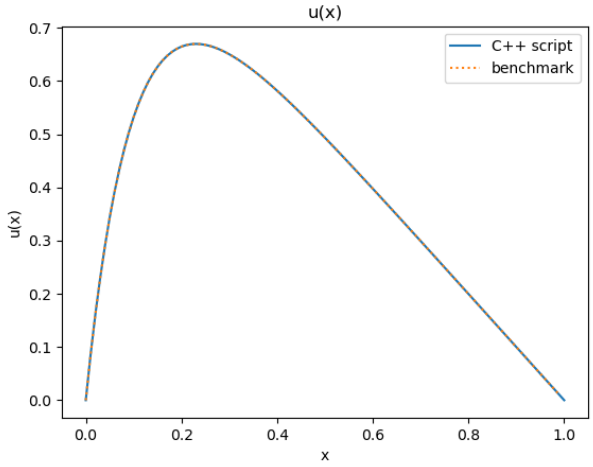

FIG. 1: The $u(x)$ generated with C++ shown alongside a Python benchmark, confirming correctness.

PROBLEM 4

Using that g_i are the elements of the discretised $f(x_i)$, we can see by Equation 1 that we can write out the elements g_i using Equation 4. This gives us, for the first three elements:

$$g_1 = \frac{1}{h^2}(v_2 - 2v_1)$$
$$g_2 = \frac{1}{h^2}(v_3 - 2v_2 + v_1)$$
$$g_3 = \frac{1}{h^2}(v_4 - 2v_3 + v_2)$$

Thus we can write the matrix equation:

$$\frac{1}{h^2}\begin{bmatrix} -2 & 1 & 0 & 0 & ... & a_{1,n} \\ 1 & -2 & 1 & 0 & ... & a_{2,n} \\ 0 & 1 & -2 & 1 & ... & a_{2,n} \\ : & & & & & \\ a_k, 1 & ... & ... & ... & ... & a_{k,n} \end{bmatrix} \cdot \begin{bmatrix} v_1 \\ v_2 \\ v_3 \\ : \\ v_n \end{bmatrix} = \begin{bmatrix} g_1 \\ g_2 \\ g_3 \\ : \\ g_n \end{bmatrix}$$

$$\begin{bmatrix} 2 & -1 & 0 & 0 & ... & a_{1,n} \\ -1 & 2 & -1 & 0 & ... & a_{2,n} \\ 0 & -1 & 2 & -1 & ... & a_{2,n} \\ : & & & & & \\ a_{k,1} & ... & ... & ... & ... & a_{k,n} \end{bmatrix} \cdot \begin{bmatrix} v_1 \\ v_2 \\ v_3 \\ : \\ v_n \end{bmatrix} = -h^2 \begin{bmatrix} g_1 \\ g_2 \\ g_3 \\ : \\ g_n \end{bmatrix} \tag{5}$$

Thus,

$$\boldsymbol{A}\vec{v} = \vec{g} \tag{6}$$

where $\boldsymbol{A}$ is a tridiagonal matrix ('... a square matrix whose nonzero entries can occur only on the main diagonal, the subdiagonal, and the superdiagonal' [3]) as shown in Equation 5, which also shows its signature as $(-1, 2, -1)$.

In terms of Equation 1, the relationship between the elements of $\vec{g}$ and the elements of $\vec{v}$ (our approximation of u) are given in the three lines above, which generalise to

$$g_i = \frac{1}{h^2}(v_{i+1} - 2v_i + v_{i-1})$$

As for the relationship between $\vec{g}$ and the right-hand side of Equation 1,

$$g_i = f(x_i)$$

PROBLEM 5

5a

In my derivation of Equation 6 I have assumed that the rows of $\boldsymbol{A}$ are given by the amount of steps that x has been discretised into, since each row will yield an entry $g_i = f(x_i)$. Looking at the formulation of this problem in context of the lecture notes [2], we can recognise that this excludes the known boundary conditions v_0 and v_{n+1}. Therefore, $\vec{v}^*$ presents the approximate solution for all x_i including the boundary conditions. Thus $n = m + 2$, where the 2 stands for the two boundary points that are known and therefore need not be approximated in $\vec{v}$

5b

See above: when we find $\vec{v}$ we find the subset of $\vec{v}^*$ that excludes the boundary points v_0 and v_{n+1}.

PROBLEM 6

6a

The algorithm consists of two steps. The first one is forward substitution to get to an upper triangular form [2]. For a matrix $\boldsymbol{A}$ with entries $a_{i,j}$, shown here for $i, j = 0, ..., n$ with $n = 4$, we get the following matrix:

$$\begin{bmatrix} a_{1,1} & a_{1,2} & 0 & 0 \\ a_{2,1} & a_{2,2} & a_{2,3} & 0 \\ 0 & a_{3,2} & a_{3,3} & a_{3,4} \\ 0 & 0 & a_{4,3} & a_{4,4} \end{bmatrix}$$

Describing each row as R_i, we can leave R_1 as-is and perform the following for the following rows:

$$\begin{aligned} R_2 &= R_2 - \frac{a_{2,1}}{a_{1,1}} R_1 \\ R_3 &= R_3 - \frac{a_{3,2}}{a_{2,2}} R_2 \\ R_4 &= R_4 - \frac{a_{4,3}}{a_{3,3}} R_3 \\ \text{Summarising: } R_i &= R_i - \frac{a_{i,i-1}}{a_{i-1,i-1}} R_{i-1} \text{ for } i = 2, ..., n \end{aligned}$$

Applying this algorithm we arrive at the upper triangular form

$$\begin{bmatrix} a_{1,1} & a_{1,2} & 0 & 0 \\ 0 & a_{2,2} & a_{2,3} & 0 \\ 0 & 0 & a_{3,3} & a_{3,4} \\ 0 & 0 & 0 & a_{4,4} \end{bmatrix}$$

The next step is back substitution, where we aim to get the matrix to the identity matrix I [2]. Starting at $R_4 = \frac{R_4}{a_{4,4}}$ we can then write an algorithm for the following rows:

$$R_3 = \frac{R_3 - a_{3,4} \cdot R_4}{a_{3,3}}$$
$$R_2 = \frac{R_2 - a_{2,3} \cdot R_3}{a_{2,2}}$$
$$R_1 = \frac{R_1 - a_{1,2} \cdot R_2}{a_{1,1}}$$
$$\text{Summarising: } R_i = \frac{R_i - a_{i,i+1} \cdot R_{i+1}}{a_{i,i}} \text{ for } i = n-1, ..., 1$$

According to the lecture notes by A. Kvellestad [2], we can simplify this matrix algorithm by defining:

1. $\vec{a}$ as the subdiagonal
2. $\vec{b}$ as the main diagonal
3. $\vec{c}$ as the superdiagonal
4. $\vec{g}$, which shows up on the right hand side of our augmented matrix as

$$\left[\begin{array}{cccc|c} b_1 & c_1 & 0 & 0 & g_1 \\ a_2 & b_2 & c_2 & 0 & g_2 \\ 0 & a_3 & b_3 & c_3 & g_3 \\ 0 & 0 & a_4 & b_4 & g_4 \end{array}\right]$$

5. vector $\tilde{b}$ for the updated values of $\vec{b}$ that result from the forward and backward substitutions
6. vector $\tilde{g}$ for the updated values of $\vec{g}$ that result from the forward and backward substitutions
7. $\vec{v}$ presenting our solution

This allows us to summarise the algorithms that I previously established for each row in terms of the vectors $\vec{a}, \vec{c}$ and both the regular and updated versions of $\vec{b}$ and $\vec{g}$ [2]:

Forward substitution:

$$\tilde{b_1} = b_1 \tag{7}$$
$$\tilde{b_i} = b_i - \frac{a_i}{\tilde{b}_{i-1}} c_{i-1} \text{ for } i = 2, ..., n \tag{8}$$
$$\tilde{g_1} = g_1 \tag{9}$$
$$\tilde{g_i} = g_i - \frac{a_i}{\tilde{b}_{i-1}} \tilde{g}_{i-1} \text{ for } i = 2, ..., n \tag{10}$$

Backward substitution:

$$v_4 = \frac{\tilde{g}_4}{\tilde{b}_4} \tag{11}$$
$$v_i = \frac{\tilde{g}_i - c_i v_{i+1}}{\tilde{b}_i} \text{ for } i = n-1, ..., 1 \tag{12}$$

6b

First we find the number of FLOPs for each line in the final set of Equations 7-12:

7. 0 FLOPs
8. 1 divide, 1 multiply, 1 minus = 3 FLOPs per i, so $3(n-1)$

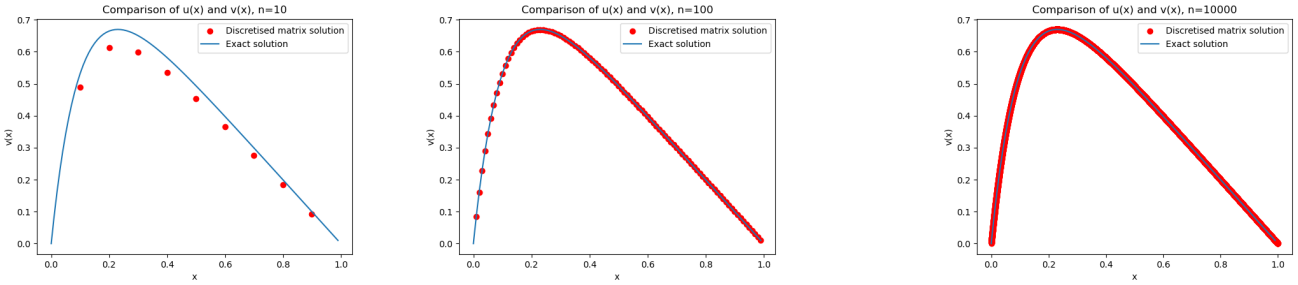

FIG. 2: $v(x)$ compared to exact solution for (a) $n = 10$, (b) $n = 100$, and (c) $n = 10000$

9. 0 FLOPs

10. 1 divide, 1 multiply, 1 minus = 3 FLOPs per i, so $3(n-1)$

11. 1 divide = 1 FLOP

12. 1 divide, 1 multiply, 1 minus = 3 FLOPs per i, so $3(n-1)$

Counting up the FLOPs per line we get to a total of $9(n-1)+1$ FLOPs.

PROBLEM 7

7a

The required algorithm can be summarised as follows:

Algorithm 1 General algorithm

Initialise vector of x of size n with step size h
Initialise vectors $a = c = -1$ of length x.size()
Initialise vector $b = 2$ of length x.size()
Initialise vector $g = h^2 100 e^{-10x}$
Initialise vectors $\tilde{b} = \tilde{g} = v = 0$ of length x.size(), to be filled as we go
Assign $\tilde{b_0}$ and $\tilde{g_0}$
for $i = 1, ..., n$ **do**
    $\tilde{b_i} = b_i - \frac{a_i}{\tilde{b}_{i-1}} c_{i-1}$
    $\tilde{g_i} = g_i - \frac{a_i}{\tilde{b}_{i-1}} \tilde{g}_{i-1}$
Compute v_n
for $i = n-1, ..., 0$ **do**
    $v_i = \frac{\tilde{g}_i - c_i v_{i+1}}{\tilde{b}_i}$

The code file containing this algorithm may be found in the Github repository.

7b

Figure 2a-c shows the solution from the matrix equation $v(x)$ plotted next to the exact solution for several step numbers. We see that increasing the step number (thus reducing the step size h) reduces the error as compared to the exact solution. This was to be expected from Equation 4, where the final term will shrink as h becomes smaller.

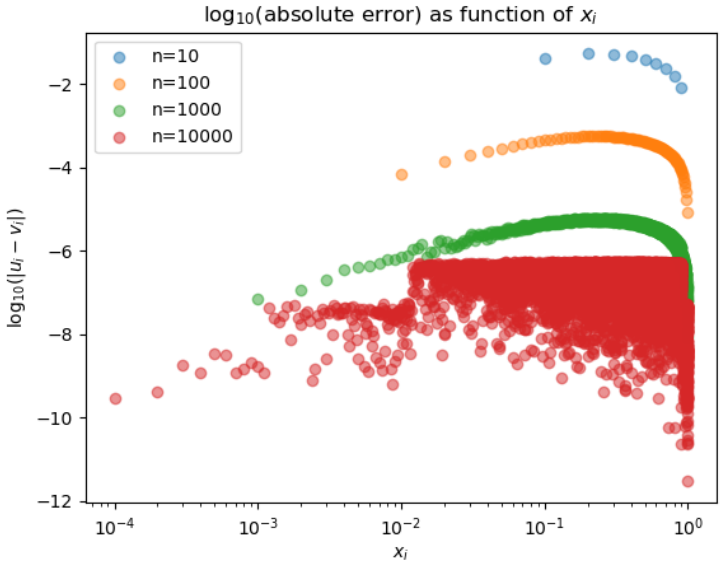

FIG. 3: Logarithm of absolute error for $v(x)$ versus exact solution, for several n.

PROBLEM 8

8a

Examining Figure 3 we see that increasing the step number causes the logarithm of the absolute error to become more negative. As we increase the step number we can expect $|u_i - v_i| \to 0$, therefore $\log(|u_i - v_i|) \to -\infty$. Illustrating this with a higher n made the plot too hard to read, therefore I stopped at $n = 10^4$.

8b

Figure 4 shows the same trend as the absolute error where error $\to 0$, hence log(error) $\to -\infty$ as the number of steps increases. For lower step sizes the relative error appears constant, but as step size increases the relative error appears to develop some dependency on the value of x_i.

8c

Figure 5 shows the maximum log of the relative error as function of the number of steps n. As expected, we see a decrease in the maximum relative error i.e. our approximation becomes more accurate as we increase n. A surprising result is the sudden spike in the error, indicating decreased accuracy, for $n = 10^7$. Due to the tiny step size, this spike can probably be attributed to a loss of precision due to roundoff errors [2].

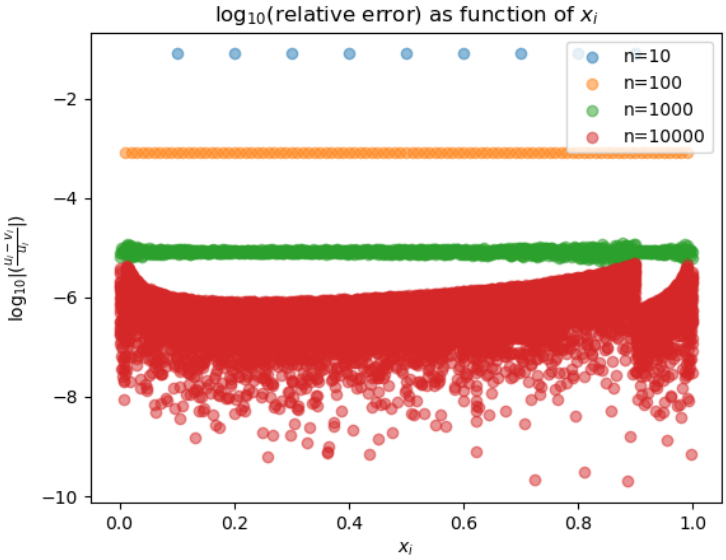

FIG. 4: Logarithm of relative error for $v(x)$ versus exact solution, for several n.

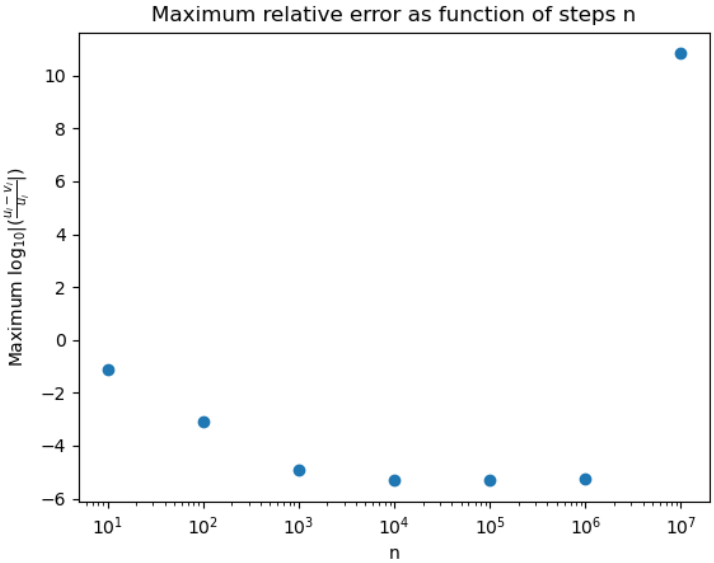

FIG. 5: Maximum log of relative error as function of number of steps n.

PROBLEM 9

9a

We can modify the General Algorithm equations to obtain the special algorithm by recognising that all elements of $\vec{a}, \vec{b}$ and $\vec{c}$ are the same as dictated by the signature of the matrix. Using that $a_i = c_i = -1$ and $b_i = 2$ we obtain:

Forward substitution:

$$\tilde{b_1} = 2 \tag{13}$$

$$\tilde{b_i} = 2 - \frac{-1}{\tilde{b}_{i-1}} \cdot -1 = 2 - \frac{1}{\tilde{b}_{i-1}} \text{ for } i = 2, ..., n \tag{14}$$

$$\tilde{g_1} = g_1 \tag{15}$$

$$\tilde{g_i} = g_i - \frac{-1}{\tilde{b}_{i-1}} \tilde{g}_{i-1} = g_i - \frac{\tilde{g}_{i-1}}{\tilde{b}_{i-1}} \text{ for } i = 2, ..., n \tag{16}$$

Backward substitution:

$$v_4 = \frac{\tilde{g}_4}{\tilde{b}_4} \tag{17}$$

$$v_i = \frac{\tilde{g}_i - (-1)v_{i+1}}{\tilde{b}_i} = \frac{\tilde{g}_i + v_{i+1}}{\tilde{b}_i} \text{ for } i = n-1, ..., 1 \tag{18}$$

9b

Going line by line as before, we can see that

13. 0 FLOPs

14. 1 divide, 1 minus = 2 FLOPs per i, so $2(n-1)$

15. 0 FLOPs

16. 1 divide, 1 minus = 2 FLOPs per i, so $2(n-1)$

17. 1 divide = 1 FLOP

18. 1 divide, 1 plus = 2 FLOPs per i, so $2(n-1)$

This brings the total to $6(n-1)+1$ FLOPs for the special algorithm, which is a reduction of $3(n-1)$ compared to the general algorithm.

9c

The required algorithm can be summarised as follows:

Algorithm 2 Special algorithm

Initialise vector of x of size n with step size h
Initialise vectors $a = c = -1$ of length x.size()
Initialise vector $b = 2$ of length x.size()
Initialise vector $g = h^2 100 e^{-10x}$
Initialise vectors $\tilde{b} = \tilde{g} = v = 0$ of length x.size(), to be filled as we go
Assign $\tilde{b_0}$ and $\tilde{g_0}$
for $i = 1, ..., n$ **do**
    $\tilde{b_i} = 2 - \frac{1}{\tilde{b}_{i-1}}$
    $\tilde{g_i} = g_i - \frac{\tilde{g}_{i-1}}{\tilde{b}_{i-1}}$
Compute v_n
for $i = n - 1, ..., 0$ **do**
    $v_i = \frac{\tilde{g}_i + v_{i+1}}{\tilde{b}_i}$

The code file containing this algorithm may be found in the Github repository.

PROBLEM 10

For the purposes of this exercise I wrote a separate code according to the following algorithm:

Algorithm 3 Timing algorithm

Initialise vector of planned step numbers
Initialise vector of vectors with time taken per step number, with 10 samples per step number
for i = lowest step number, ..., highest step number **do**
    for $j = 0, ..., 10$ **do**
        Start timer
        General Algorithm
        Stop timer
        Timerecords[j][i] = time taken
Write out recorded times to file
Repeat with Special Algorithm

The resulting code file can be found in my Github repository. It was used to generate the results shown in Figure 6, displaying the time taken for both algorithms as function of the step size. One would generally expect the special algorithm to become faster than the general algorithm as the step number grows (i.e. the step size shrinks) due to the lower number of FLOPs in the special as compared to the general algorithm. Figure 6 a shows the time taken on a regular scale, highlighting the exponential nature of the time taken as function of step size. It also shows the increasing spread in the time taken as the step size grows smaller. Figure 6b is the same figure but with a log scale on the y axis to show the results for the bigger step sizes more clearly. The special algorithm being faster than the general is not really evident from these figures. We can see some evidence of it when looking at the mean of the time taken across all samples in Figure 7a. Looking at the standard deviation across samples for each algorithm in Figure 7b we also see that the time taken for the special algorithm is more stable than for the general algorithm as step size shrinks.

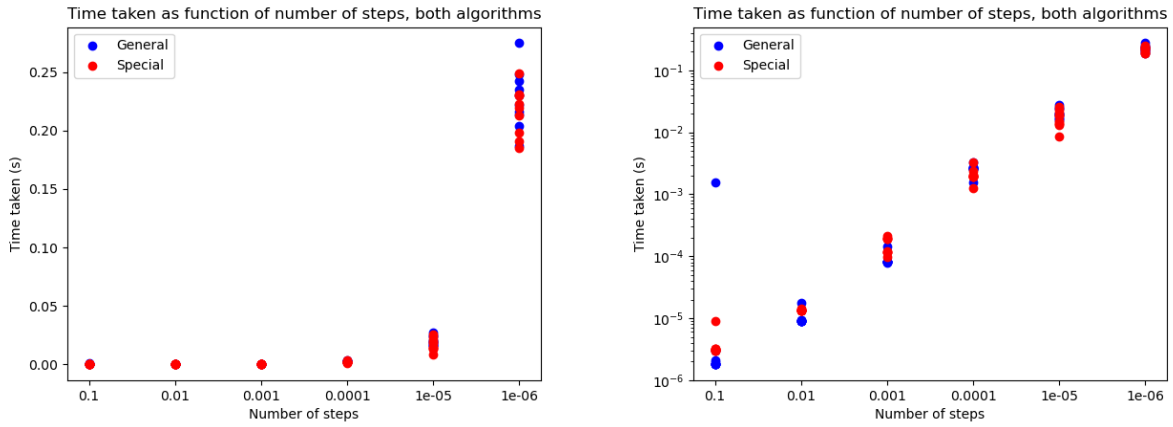

FIG. 6: Time taken for both algorithms to run as function of the step size (a) with a regular y axis and (b) with a log scale on the y axis.

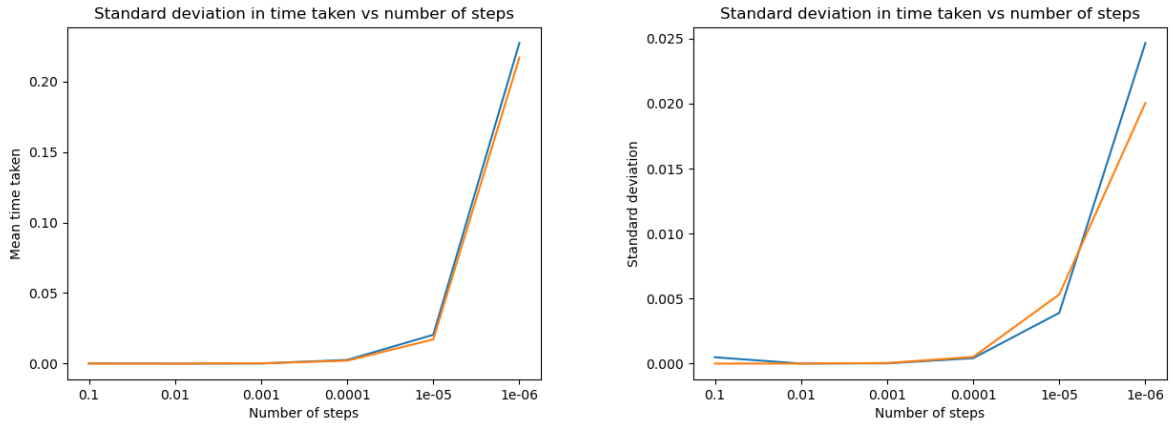

FIG. 7: (a) mean time taken and (b) standard deviation in the time taken for each algorithm as function of the step size.

DECLARATION ON THE USE OF AI

AI was used in one instance in this project: when the code written in 7a continued to fail to reproduce the expected result, even after extensive debugging, the code and equations it was intended to contain were given to ChatGPT for debugging. ChatGPT spotted a small mistake in one of the formulae (g instead of $\tilde{g}$). See also the issue that I wrote at **https://github.uio.no/anderkve/FYS3150-forum/issues/100**. My full interaction with ChatGPT in this instance is included in the folder 'AI logs' in the Github repository. All codes and text in this project were self-written using the sources given in the bibliography.

REFERENCES

[1] Y. D. Chong. *7.2: Discretizing Partial Differential Equations — phys.libretexts.org*. https://phys.libretexts.org/{B}ookshelves/{M}athematical_{P}hysics_and_{P}edagogy/{C}omputational_{P}hysics_({C}hong)/07%3{A}_{F}inite-{D}ifference_{E}quations/7.02%3{A}_{D}iscretizing_{P}artial_{D}ifferential_{E}quations. [Accessed 09-09-2026].

[2] A. Kvellestad. Lecture notes for FYS3150. URL: https://github.com/anderkve/FYS3150/tree/master/lecture_notes/2026.

[3] Wolfram Mathworld. *Tridiagonal Matrix – from Wolfram MathWorld — mathworld.wolfram.com*. https://mathworld.wolfram.com/{T}ridiagonal{M}atrix.html. [Accessed 10-09-2026].